SPLIT STRINGS INTO UNIQUE PRIMES

Company: Amazon
Round: Online Assessment
Year: 2021
Difficulty: Medium
Topic: Dynamic Programming, Prime Numbers, String
Source: https://algo.monster/problems/amazon_oa_num_ways_to_split_into_primes

Problem Description:

We are given a string containing only digits from 0 to 9.

Our task is to split this string into smaller parts so that:

1. Every part represents a prime number.
2. Every prime number must be between 2 and 1000.
3. A part cannot start with 0.
4. We must use every digit in the string.
5. We need to find the total number of different ways in which the string can be split.

Input Format:

The input is a string containing digits.

Output Format:

Return the number of different ways in which the complete string can be split into valid prime numbers.

Sample Input:

String s = "31173"

Sample Output:

Number of Ways to Split = 6

Explanation:

Understanding the Problem

Let's first understand what makes a split valid.

Suppose we have:

31173

We can choose different lengths for each part.

For example:

3 | 11 | 7 | 3

or:

31 | 17 | 3

or:

311 | 73

But every part must satisfy the rules.

For example:

4

cannot be used because 4 is not prime.

Similarly:

1013

cannot be used because it is greater than 1000.

What Is a Prime Number?

A prime number is a number greater than 1 that can only be divided by:

- 1
- itself

Some examples are:

2, 3, 5, 7, 11, 13, 17, 19...

Numbers such as:

4, 6, 8, 9, 10

are not prime.

In this problem, we only care about prime numbers from:

2 to 1000

Important Rule: No Leading Zero

A prime number cannot start with 0.

For example:

02

cannot be considered a valid part.

Even though 02 has the value 2, the problem does not allow a prime number to start with zero.

So whenever we see a 0 at the beginning of a part, we simply don't consider that split.

Why Is 1000 Important?

Every part must be at most 1000.

That means a part can have at most 4 digits.

For example:

997

can be considered.

But:

1009

cannot be considered because it is greater than 1000.

So while splitting the string, we only need to check at most 4 digits at a time.

This makes the problem much easier and faster.

The string:

31173

can be divided into prime numbers in the following 6 ways:

Way 1

3 | 11 | 7 | 3

Way 2

3 | 11 | 73

Way 3

31 | 17 | 3

Way 4

31 | 173

Way 5

311 | 7 | 3

Way 6

311 | 73

Every part in these six cases is a prime number between 2 and 1000.

Therefore:

Answer = 6

Approach to Solve This Problem:

A simple way to solve this problem is using Dynamic Programming (DP)**.**

The main idea is:

Instead of trying every possible splitting from the beginning again and again, we remember how many valid ways we have already found for each position.

Step 1: Create a DP Array

Let:

dp[i]

mean:

The number of valid ways to split the first i digits of the string.

For example, if:

s = "31173"

then:

dp[0]

means the number of ways to split an empty string.

There is one way to do that: choose nothing.

So:

dp[0] = 1

This starting value helps us build all the other answers.

Step 2: Look at Possible Prime Numbers

Starting from each position, we try to take:

- 1 digit
- 2 digits
- 3 digits
- 4 digits

Why only 4?

Because the largest allowed number is 1000.

For example, from:

31173

starting at the first position, we can check:

3
31
311
3117

But 3117 is greater than 1000, so we stop.

Step 3: Check Whether the Number Is Prime

For every number we create, we check:

1. Does it start with 0?
2. Is it between 2 and 1000?
3. Is it prime?

If all three conditions are satisfied, it can be used as one part of the split.

Step 4: Update DP

Suppose we are at position i and we find a valid prime using the digits from i to j.

Then the number of ways to reach j + 1 increases by the number of ways to reach i.

In simple terms:

If there are already 5 ways to reach this position, and we find a valid prime from here, those 5 ways can all continue using that prime.

So we do:

dp[j + 1] += dp[i]

Simple Example of DP

Suppose we have:

"23"

We start with:

dp[0] = 1

From position 0, we can take:

2

2 is prime, so:

dp[1] += dp[0]

Therefore:

dp[1] = 1

Now from position 1, we can take:

3

which is also prime.

So:

dp[2] += dp[1]

giving:

dp[2] = 1

We can also take:

23

directly because 23 is prime.

So there is another way:

2 | 3
23

Therefore the answer is:

2

Why Dynamic Programming Works

Without DP, we might repeatedly check the same parts of the string.

For example, if several different splits reach the same position, we don't need to solve the remaining part again and again.

We simply remember:

dp[i] = number of ways to reach this position

and use that information.

This makes the solution much more efficient.

Approach in Simple Steps

The complete approach is:

Start from the beginning
↓
Try taking 1 to 4 digits
↓
Check if the number is a valid prime
↓
If valid, update dp
↓
Move to the next position
↓
Repeat until the entire string is used

↓

Return dp[n]

where n is the length of the string.

Java Code:

Main.java > Main

```java
import java.util.*;

public class Main {

    // Check whether a number is prime
    public static boolean isPrime(int num) {

        if (num < 2) {
            return false;
        }

        for (int i = 2; i * i <= num; i++) {

            if (num % i == 0) {
                return false;
            }
        }

        return true;
    }

    public static int countWays(String s) {

        int n = s.length();

        // dp[i] = number of ways to split first i characters
        int[] dp = new int[n + 1];

        // One way to split an empty string
        dp[0] = 1;

        for (int i = 0; i < n; i++) {

            // If there are no ways to reach this position,
            // there is nothing to do
            if (dp[i] == 0) {
                continue;
            }
```

```
        // A prime number can have at most 4 digits
        int number = 0;

        for (int j = i; j < n && j < i + 4; j++) {

            // A number cannot start with zero
            if (j > i && s.charAt(i) == '0') {
                break;
            }

            number = number * 10 + (s.charAt(j) - '0');

            // Number must be between 2 and 1000
            if (number >= 2 && number <= 1000
                    && isPrime(number)) {

                dp[j + 1] += dp[i];
            }

            // No need to continue after 1000
            if (number > 1000) {
                break;
            }
        }
    }

    return dp[n];
}
```

```
    Run | Debug
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // Input string
        String s = sc.next();

        int answer = countWays(s);

        System.out.println(answer);

        sc.close();
    }
}
```

Solution:

The solution uses Dynamic Programming.We keep track of how many valid ways can reach every position in the string.At each position, we try to take a number of 1 to 4 digits because every valid prime must be between 2 and 1000.

For every number:

- If it starts with 0, we reject it.
- If it is greater than 1000, we stop.
- If it is prime, we update the DP array.

At the end:

dp[n]

contains the number of ways to split the entire string into valid prime numbers.

For:

31173

the final answer is:

6

Understanding the Important Part of the Code:

1. isPrime() Function

public static boolean isPrime(int num)

This function checks whether a number is prime.

We try dividing the number by possible divisors.

if (num % i == 0)

If it divides exactly, the number is not prime.

If no divisor is found, the number is prime.

2. DP Array

int[] dp = new int[n + 1];

dp[i] stores:

How many ways can we split the first i characters?

We start with:

dp[0] = 1;

because there is one starting point before we process any digits.

3. Trying 1 to 4 Digits

```
for (int j = i; j < n && j < i + 4; j++)
```

We check at most 4 digits because valid numbers cannot be greater than 1000.

For example, we try:

3
31
311
3117

and stop when the number becomes greater than 1000.

4. Creating the Number

```
number = number * 10 + (s.charAt(j) - '0');
```

This builds the number digit by digit.

For example, for:

311

we build it like this:

3
3 × 10 + 1 = 31
31 × 10 + 1 = 311

5. Checking the Prime

```
if (number >= 2 && number <= 1000
      && isPrime(number))
```

This makes sure that the number:

- is at least 2

- is at most 1000
- is prime

Only then do we accept it as a part of the split.

6.Updating DP

dp[j + 1] += dp[i];

This means:

All the valid ways that reached position i can now continue with this prime number.

This is the main DP step.

Complexity Analysis

Time Complexity: O(n)

At every position, we check at most 4 digits.Checking whether a number up to 1000 is prime also takes only a small constant amount of work.

Therefore, overall:

Time Complexity = O(n)

Space Complexity: O(n)

We use a DP array of size n + 1.

Therefore:

Space Complexity = O(n)